

## VII - TP 2

## Algorithmes de tri

## 1 Introduction

Dans ce TP, on cherche à implémenter des algorithmes qui permettent de trier un tableau. On considérera que tous les tableaux sont non vides (sinon il n'y a rien à trier !). On peut trier un tableau de deux manières différentes : soit par ordre croissant, soit par ordre décroissant : lorsque l'on dira "le tableau est trié" dans ce TP, on sous-entendra qu'il est trié par ordre croissant. On commence par écrire le code de certaines fonctions qui seront utiles par la suite.

**Question 1.** Écrire le code de la fonction `genere_tableau` prend en arguments un entier `n` et deux entiers `x_min` et `x_max` et qui renvoie un tableau de `n` entiers tirés au hasard compris entre `x_min` et `x_max` inclus.

On utilisera pour cela la fonction `randint` du module `random`, dont on donne un exemple d'utilisation ci-dessous. `randint(x, y)` renvoie un entier choisi aléatoirement entre `x` et `y` inclus.

Code python

```
1 from random import randint
2
3 print(randint(-5, 5))
4 print(randint(-5, 5))
5 print(randint(-5, 5))
```

 Résultat

```
3
-5
1
```

Code python

```
1 def genere_tableau(n, x_min, x_max):
2     """ int, int, int -> [int]
3     Renvoie un tableau d'entiers tirés au hasard parmi [x_min, x_max] """
4     pass
```

Code python

```
1 print(genere_tableau(5, -10, 10))
```

 Résultat

```
[7, 7, 6, 7, -7]
```

**Question 2.** Écrire le code de la fonction `est_trie` qui prend en argument un tableau `tab` et qui renvoie `True` si le tableau est trié par ordre croissant, `False` sinon.

Pour cela, on parcourra tous les éléments du tableau les uns après les autres : si on trouve un élément d'indice `i` tel que l'élément d'indice `i + 1` lui est inférieur, alors cela signifie que le tableau n'est pas trié par ordre croissant. Au contraire, si on ne trouve pas un tel élément, cela signifie que le tableau est trié par ordre croissant.

Code python

```
1 def est_trie(tab):
2     """ [int] -> bool
3     Renvoie True si tab est trié par ordre croissant, False sinon """
4     pass
```

Code python

```
1 print(est_trie([1]))
2 print(est_trie([2, 4, 6]))
3 print(est_trie([6, 4, 2]))
4 print(est_trie([1, 3, 2]))
```

 Résultat

```
True
True
False
False
```

**Question 3.** Écrire le code de la fonction `echange` qui prend en argument un tableau `tab` et deux indices `i` et `j` compatibles avec la taille du tableau. Cette fonction modifie `tab` en place et échange la position des éléments d'indice `i` et `j` dans `tab`.

Code python

```
1 def echange(tab, i, j):
2     """ [int], int, int -> None
3     Échange les éléments d'indice i et j dans tab """
4     pass
```

Code python

```
1 tab = [6, 7, 8]
2 echange(tab, 0, 2)
3 print(tab)
4 echange(tab, 1, 0)
5 print(tab)
```

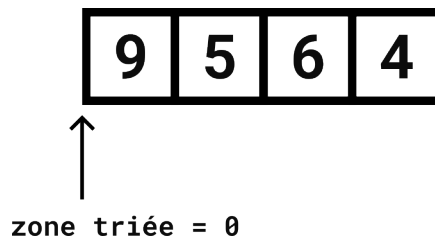
⚙️ ➡ Résultat

```
[8, 7, 6]
[7, 8, 6]
```

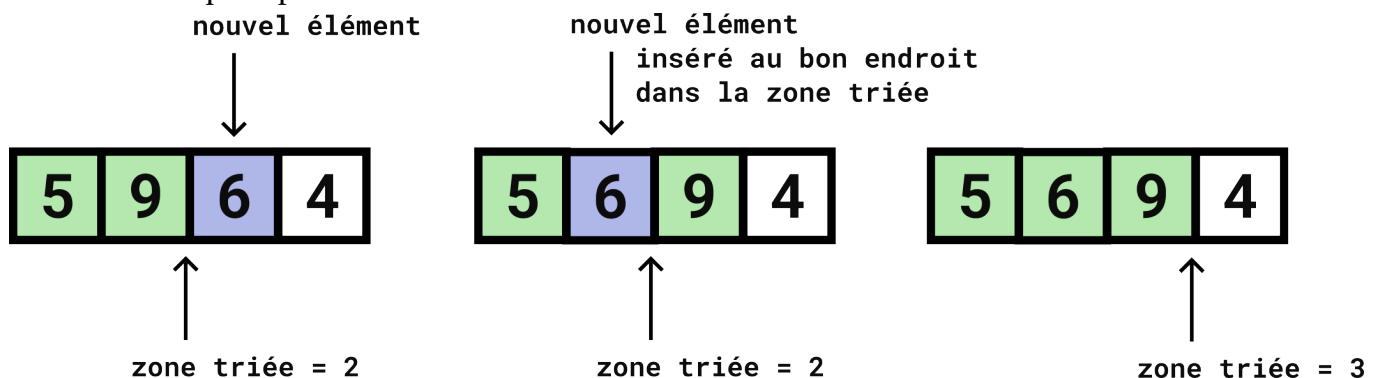
## 2 Tri par insertion

On cherche à implémenter l'algorithme du tri par insertion. L'idée de cet algorithme est la suivante :

- on maintient une zone triée d'éléments tous situés au début du tableau. Initialement cette zone est vide et contient 0 élément.



- tant que tous les éléments du tableau ne se trouvent pas dans la zone triée :
  - on choisit un élément en dehors de cette zone triée (habituellement l'élément du tableau qui se situe juste après le dernier élément de la zone triée) ;
  - on l'**insère** dans la zone triée en réalisant des échanges successifs avec l'élément précédent dans le tableau. On réalise ces échanges tant que le nouvel élément n'est pas le premier élément du tableau et si le nouvel élément est plus petit que l'élément qui le précède.



Après chaque insertion dans la zone triée du tableau, celle-ci est constituée d'un élément de plus. Si  $n$  est la taille du tableau et que l'on réalise  $n$  insertions, alors à l'issue de cet algorithme le tableau est complètement trié.

**Question 4.** 1. Jouer au jeu du tri par insertion à l'adresse :

[https://www.advanced-ict.info/interactive/insertion\\_sort.html](https://www.advanced-ict.info/interactive/insertion_sort.html)

Vous ne devez pas faire d'erreur.

2. Compléter le code de la fonction `tri_insertion` qui prend en argument un tableau `tab` et qui le trie en place à l'aide de l'algorithme du tri par insertion.

Code python

```
1 def tri_insertion(tab):
2     """ [int] -> None
3     Trie le tableau en place. """
4     # Si le tableau est vide il n'y a rien à faire.
5     if len(tab) == 0:
6         return
7     zone_triee = 0
8     # Tant qu'il y a des éléments à trier dans le tableau
9     while ...:
10         pos_nouveau = zone_triee
11         nouveau_elem = tab[zone_triee]
12         # On échange le nouvel élément avec
13         # l'élément du tableau qui se trouve
14         # à sa gauche tant que cela est nécessaire
15         while ...:
16             echange(tab, ..., ...)
17             pos_nouveau = ... # on met à jour la position du nouvel élément
18             # Il y a un élément de plus dans la zone triée
19             zone_triee = ...
```

Code python

```
1 tab = [9, 5, 6, 4, -3, 0, 8, 1]
2 tri_insertion(tab)
3 print(tab)
```

 Résultat

[-3, 0, 1, 4, 5, 6, 8, 9]

**Question 5.** À l'aide des fonctions `genere_tableau` et `est_trie` et de l'instruction `assert` écrire un code python qui génère aléatoirement 10 tableau de taille 20 constitués d'entiers compris entre -30 et 30 et qui vérifie que la fonction `tri_insertion` tri correctement chacun de ces tableaux.

### 3 Tri à bulle

On cherche à implémenter l'algorithme du tri à bulles. L'idée de cet algorithme est la suivante. Si  $n$  est le nombre d'éléments du tableau, on réalise  $n$  passages : on parcourt le tableau de gauche à droite. Si deux éléments consécutifs ne sont pas rangés dans l'ordre croissant, alors on échange leurs positions.

À la fin du premier passage le plus grand élément du tableau se trouve à la fin du tableau : il est correctement placé. À la fin du deuxième passage, le deuxième plus grand élément se trouve à l'avant dernière position du tableau : il est correctement placé. De même, à la fin du  $n$ -ième passage le plus petit élément se trouve au début du tableau : il est correctement placé.

**Question 6.** 1. Jouer au jeu du tri à bulle à l'adresse :

[https://www.advanced-ict.info/interactive/bubble\\_sort.html](https://www.advanced-ict.info/interactive/bubble_sort.html)

Vous ne devez pas faire d'erreur.

2. Compléter le code de la fonction `tri_bulles` qui prend en argument un tableau `tab` et qui le trie en place à l'aide de l'algorithme du tri à bulles.

Code python

```
1 def tri_bulles(tab):
2     """ [int] -> None
3     Trie le tableau en place. """
4     if len(tab) == 0:
5         return
6     for passage in range(...):
7         for i in range(...):
8             # si deux éléments consécutifs ne sont
9             # pas bien ordonnés, on échange leur position
10            if ...:
11                échange(...)
```

Code python

```
1 tab = [9, 5, 6, 4, -3, 0, 8, 1]
2 tri_bulles(tab)
3 print(tab)
```

⚙️ ➤ Résultat

[-3, 0, 1, 4, 5, 6, 8, 9]

**Question 7.** À l'aide des fonctions `genere_tableau` et `est_trie` et de l'instruction `assert` écrire un code python qui génère aléatoirement 10 tableau de taille 20 constitués d'entiers compris entre -30 et 30 et qui vérifie que la fonction `tri_bulles` tri correctement chacun de ces tableaux.

## 4 Tri par sélection

On cherche à implémenter l'algorithme du tri par sélection, qui présente un fonctionnement similaire au tri à bulle : plutôt que d'échanger deux éléments consécutifs, on fait directement remonter le maximum "au bon endroit". Ainsi, on réalise  $n$  passages, et, lors du  $i$ -ième passage, on calcule le maximum parmi les  $n - i + 1$  premiers éléments du tableau. On échange ce maximum avec le  $(n - i + 1)$ -ième élément du tableau.

**Question 8.** 1. Écrire le code de la fonction `maximum_parmi` qui prend en argument un tableau `tab` et un entier `i` qui renvoie l'indice du plus grand élément de `tab` parmi les `i` premiers éléments de `tab`.

2. Compléter le code de la fonction `tri_selection` qui prend en argument un tableau `tab` et qui le trie en place à l'aide de l'algorithme du tri par sélection.

Code python

```
1 def maximum_parmi(tab, i):
2     """ [int], int -> int
3     Renvoie l'indice du maximum
4     ↪ parmi les n-i+1 premiers
5     ↪ éléments de tab """
6     pass
7
8 def tri_selection(tab):
9     """ [int] -> None """
10    for i in range(...):
11        i_maxi = ...
12        échange(...)
```

Code python

```
1 tab = [9, 5, 6, 4, -3, 0, 8, 1]
2 tri_selection(tab)
3 print(tab)
```

⚙️ ➤ Résultat

[-3, 0, 1, 4, 5, 6, 8, 9]